Лабораторная работа №3: Daisyworld

Федорова Анжелика Игоревна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Ход работы **7**

3.1 1. Подготовка проекта 7
3.2 2. Реализация модели 7
3.3 3. Базовая визуализация 16
3.4 4. Анимация модели 21
3.5 5. Динамика числа маргариток 22
3.6 6. Динамика модели 24
3.7 7. Параметрические эксперименты 26

4 Общий анализ **34**

5 Подтверждение выполнения (скриншоты) **35**

5.1 Установка зависимостей 35
5.2 Запуск модели 36
5.3 Генерация анимации 36
5.4 Построение графика численности 36
5.5 Построение комплексного графика 37
5.6 Параметрические эксперименты 38

6 Выводы **40**

7 Список литературы **41**

Список иллюстраций

Список таблиц

1 Цель работы

Реализовать агентную модель Daisyworld, провести вычислительные эксперименты и исследовать поведение системы при различных параметрах.

2 Задание

В ходе работы необходимо:

1. Создать структуру проекта
2. Установить необходимые пакеты
3. Реализовать модель Daisyworld
4. Выполнить визуализацию
5. Построить графики динамики
6. Провести параметрические эксперименты
7. Оформить отчёт

3 Ход работы

3.1 1. Подготовка проекта

Была создана структура проекта с использованием `DrWatson`:

- `src/` — код модели
- `scripts/` — исполняемые скрипты
- `plots/` — результаты

Установлены пакеты:

```
using Pkg
Pkg.add(["Agents", "CairoMakie", "DataFrames", "StatsBase", "Plots", "DrWatson"])
```

3.2 2. Реализация модели

Модель реализована в файле:

```
using Agents
import StatsBase
using Random

@agent struct Daisy(GridAgent{2})
    breed::Symbol
```

```
    age::Int
    albedo::Float64 # 0-1 fraction
end

function update_surface_temperature!(pos, model)
    absorbed_luminosity = if isempty(pos, model) # no daisy
        (1 - model.surface_albedo) * model.solar_luminosity
    else
        daisy = model[id_in_position(pos, model)]
        (1 - daisy.albedo) * model.solar_luminosity
    end
    local_heating = absorbed_luminosity > 0 ? 72 * log(absorbed_luminosity) + 80 : 80
    model.temperature[pos...] = (model.temperature[pos...] + local_heating) / 2
end

function diffuse_temperature!(pos, model)
    ratio = model.ratio # diffusion ratio
    npos = nearby_positions(pos, model)
    model.temperature[pos...] =
        (1 - ratio) * model.temperature[pos...] +
        sum(model.temperature[p...] for p in npos) * 0.125 * ratio
end

function propagate!(pos, model)
    isempty(pos, model) && return
    daisy = model[id_in_position(pos, model)]
    temperature = model.temperature[pos...]
    seed_threshold = (0.1457 * temperature - 0.0032 * temperature^2) - 0.6443
```

```
    if rand(abmrng(model)) < seed_threshold
        empty_near_pos = random_nearby_position(pos, model, 1, npos -> isempty(npos,
        if !isnothing(empty_near_pos)
            add_agent!(empty_near_pos, model, daisy.breed, 0, daisy.albedo)
        end
    end
end

function daisy_step!(agent::Daisy, model)
    agent.age += 1
    agent.age ≥ model.max_age && remove_agent!(agent, model)
end

function daisyworld_step!(model)
    for p in positions(model)
        update_surface_temperature!(p, model)
        diffuse_temperature!(p, model)
        propagate!(p, model)
    end
    model.tick = model.tick + 1
    solar_activity!(model)
end

function solar_activity!(model)
    if model.scenario == :ramp
        if model.tick > 200 && model.tick ≤ 400
            model.solar_luminosity += model.solar_change
        end
```

```
        if model.tick > 500 && model.tick ≤ 750
            model.solar_luminosity -= model.solar_change / 2
        end
    elseif model.scenario == :change
        model.solar_luminosity += model.solar_change
    end
end

using Random

function daisyworld(;
    griddims = (30, 30),
    max_age = 25,
    init_white = 0.2, # % cover of the world surface of white breed
    init_black = 0.2, # % cover of the world surface of black breed
    albedo_white = 0.75,
    albedo_black = 0.25,
    surface_albedo = 0.4,
    solar_change = 0.005,
    solar_luminosity = 1.0, # initial luminosity
    scenario = :default,
    seed = 165,
)

    rng = MersenneTwister(seed)
    space = GridSpaceSingle(griddims)
    properties = (;max_age, surface_albedo, solar_luminosity, solar_change, scenario,
        tick = 0, ratio = 0.5, temperature = zeros(griddims)
```

```
    )
    properties = Dict(k=>v for (k,v) in pairs(properties))

    model = StandardABM(Daisy, space; properties, rng, agent_step! = daisy_step!, mod

    grid = collect(positions(model))
    num_positions = prod(griddims)
    white_positions =
        StatsBase.sample(grid, Int(init_white * num_positions); replace = false)
    for wp in white_positions
        add_agent!(wp, Daisy, model, :white, rand(abmrng(model), 0:max_age), albedo_w
    end
    allowed = setdiff(grid, white_positions)
    black_positions =
        StatsBase.sample(allowed, Int(init_black * num_positions); replace = false)
    for bp in black_positions
        add_agent!(bp, Daisy, model, :black, rand(abmrng(model), 0:max_age), albedo_b
    end

    for p in positions(model)
        update_surface_temperature!(p, model)
    end

    return model
end
```

Описание:

- агенты: маргаритки (чёрные и белые)

- среда: сетка 30×30
- реализованы процессы:
  - нагрев поверхности
  - диффузия температуры
  - размножение
  - старение

Подключение модели:

```
using Agents
import StatsBase
using Random

@agent struct Daisy(GridAgent{2})
    breed::Symbol
    age::Int
    albedo::Float64 # 0-1 fraction
end

function update_surface_temperature!(pos, model)
    absorbed_luminosity = if isempty(pos, model) # no daisy
        (1 - model.surface_albedo) * model.solar_luminosity
    else
        daisy = model[id_in_position(pos, model)]
        (1 - daisy.albedo) * model.solar_luminosity
    end
    local_heating = absorbed_luminosity > 0 ? 72 * log(absorbed_luminosity) + 80 : 80
    model.temperature[pos...] = (model.temperature[pos...] + local_heating) / 2
end
```

```julia
function diffuse_temperature!(pos, model)
    ratio = model.ratio # diffusion ratio
    npos = nearby_positions(pos, model)
    model.temperature[pos...] =
        (1 - ratio) * model.temperature[pos...] +
        sum(model.temperature[p...] for p in npos) * 0.125 * ratio
end

function propagate!(pos, model)
    isempty(pos, model) && return
    daisy = model[id_in_position(pos, model)]
    temperature = model.temperature[pos...]
    seed_threshold = (0.1457 * temperature - 0.0032 * temperature^2) - 0.6443
    if rand(abmrng(model)) < seed_threshold
        empty_near_pos = random_nearby_position(pos, model, 1, npos -> isempty(npos,
        if !isnothing(empty_near_pos)
            add_agent!(empty_near_pos, model, daisy.breed, 0, daisy.albedo)
        end
    end
end

function daisy_step!(agent::Daisy, model)
    agent.age += 1
    agent.age ≥ model.max_age && remove_agent!(agent, model)
end

function daisyworld_step!(model)
    for p in positions(model)
```

```
        update_surface_temperature!(p, model)
        diffuse_temperature!(p, model)
        propagate!(p, model)
    end
    model.tick = model.tick + 1
    solar_activity!(model)
end

function solar_activity!(model)
    if model.scenario == :ramp
        if model.tick > 200 && model.tick ≤ 400
            model.solar_luminosity += model.solar_change
        end
        if model.tick > 500 && model.tick ≤ 750
            model.solar_luminosity -= model.solar_change / 2
        end
    elseif model.scenario == :change
        model.solar_luminosity += model.solar_change
    end
end

using Random

function daisyworld(;
    griddims = (30, 30),
    max_age = 25,
    init_white = 0.2, # % cover of the world surface of white breed
    init_black = 0.2, # % cover of the world surface of black breed
```

```
    albedo_white = 0.75,
    albedo_black = 0.25,
    surface_albedo = 0.4,
    solar_change = 0.005,
    solar_luminosity = 1.0, # initial luminosity
    scenario = :default,
    seed = 165,
)

    rng = MersenneTwister(seed)
    space = GridSpaceSingle(griddims)
    properties = (;max_age, surface_albedo, solar_luminosity, solar_change, scenario,
        tick = 0, ratio = 0.5, temperature = zeros(griddims)
    )
    properties = Dict(k=>v for (k,v) in pairs(properties))

    model = StandardABM(Daisy, space; properties, rng, agent_step! = daisy_step!, mod

    grid = collect(positions(model))
    num_positions = prod(griddims)
    white_positions =
        StatsBase.sample(grid, Int(init_white * num_positions); replace = false)
    for wp in white_positions
        add_agent!(wp, Daisy, model, :white, rand(abmrng(model), 0:max_age), albedo_w
    end
    allowed = setdiff(grid, white_positions)
    black_positions =
        StatsBase.sample(allowed, Int(init_black * num_positions); replace = false)
```

```
    for bp in black_positions
        add_agent!(bp, Daisy, model, :black, rand(abmrng(model), 0:max_age), albedo_b
    end

    for p in positions(model)
        update_surface_temperature!(p, model)
    end

    return model
end
```

3.3 3. Базовая визуализация

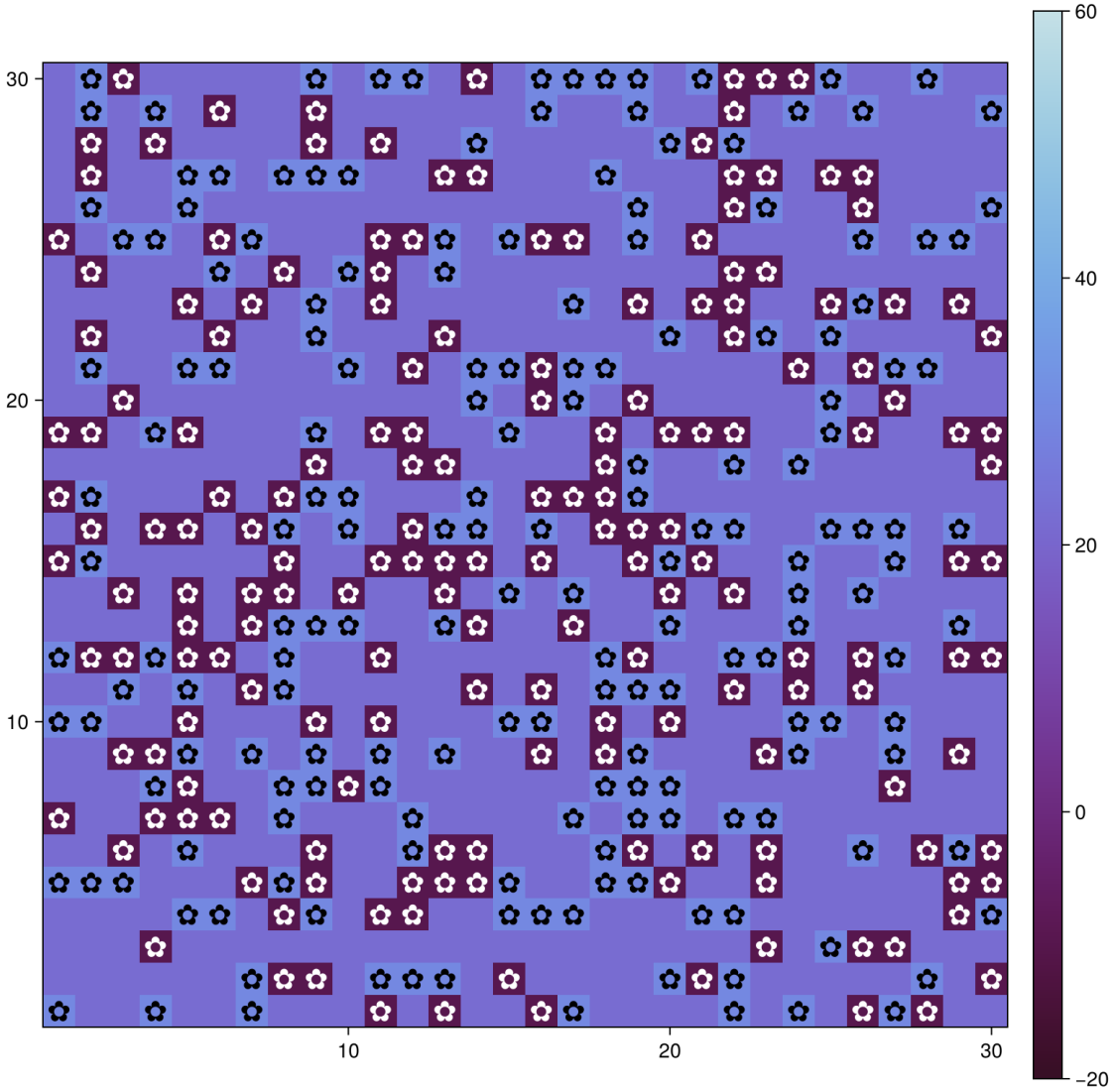
Скрипт:

```
using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots



include(srcdir("daisyworld.jl"))

using CairoMakie
model = daisyworld()

daisycolor(a::Daisy) = a.breed
```

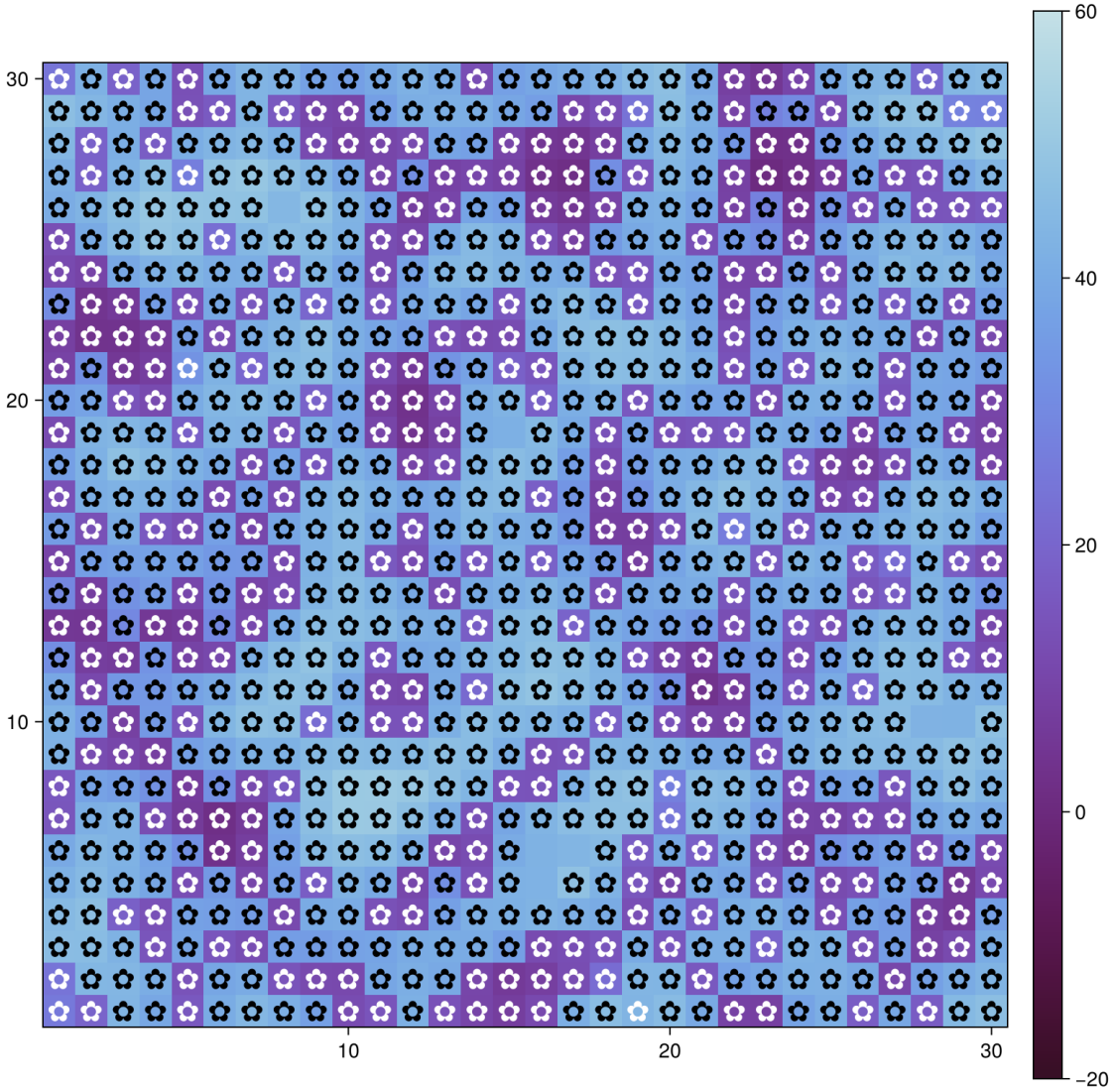
```
plotkwargs = (
    agent_color=daisycolor, agent_size = 20, agent_marker = '□',
    heatarray = :temperature,
    heatkwargs = (colorrange = (-20, 60),),
)
plt1, _ = abmplot(model; plotkwargs...)

step!(model, 5)
plt2, _ = abmplot(model; heatarray = model.temperature, plotkwargs...)

step!(model, 40)
plt3, _ = abmplot(model; heatarray = model.temperature, plotkwargs...)


save(plotsdir("daisy_step001.png"), plt1)
save(plotsdir("daisy_step005.png"), plt2)
save(plotsdir("daisy_step040.png"), plt3)
```

Результаты:

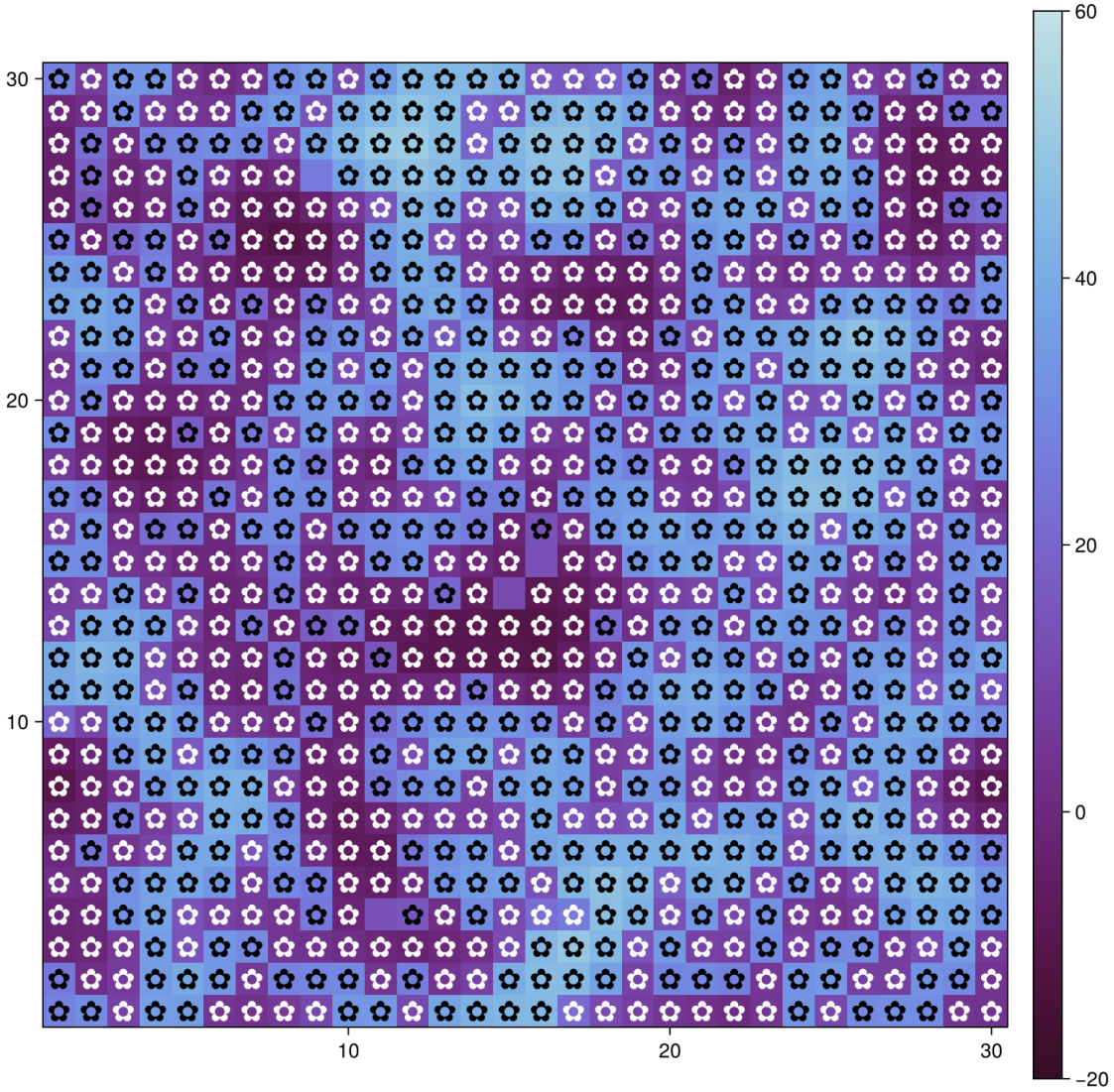
3.3.1 Шаг 1



3.3.2 Шаг 5



3.3.3 Шаг 40



Анализ:

- начальное распределение случайное
- происходит кластеризация маргариток
- формируется устойчивая структура

3.4 4. Анимация модели

Скрипт:

```
using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots

include(srcdir("daisyworld.jl"))

using CairoMakie
model = daisyworld()

daisycolor(a::Daisy) = a.breed

plotkwargs = (
    agent_color=daisycolor, agent_size = 20, agent_marker = '□',
    heatarray = :temperature,
    heatkwargs = (colorrange = (-20, 60),),
)

abmvideo(
    plotsdir("simulation.mp4"),
    model;
    title = "Daisy World",
    frames = 60,
    plotkwargs...,
```

```
)
```

Результат загружен на видеохостинг.

Анализ:

- наблюдается динамическое равновесие
- популяции колеблются
- температура стабилизируется

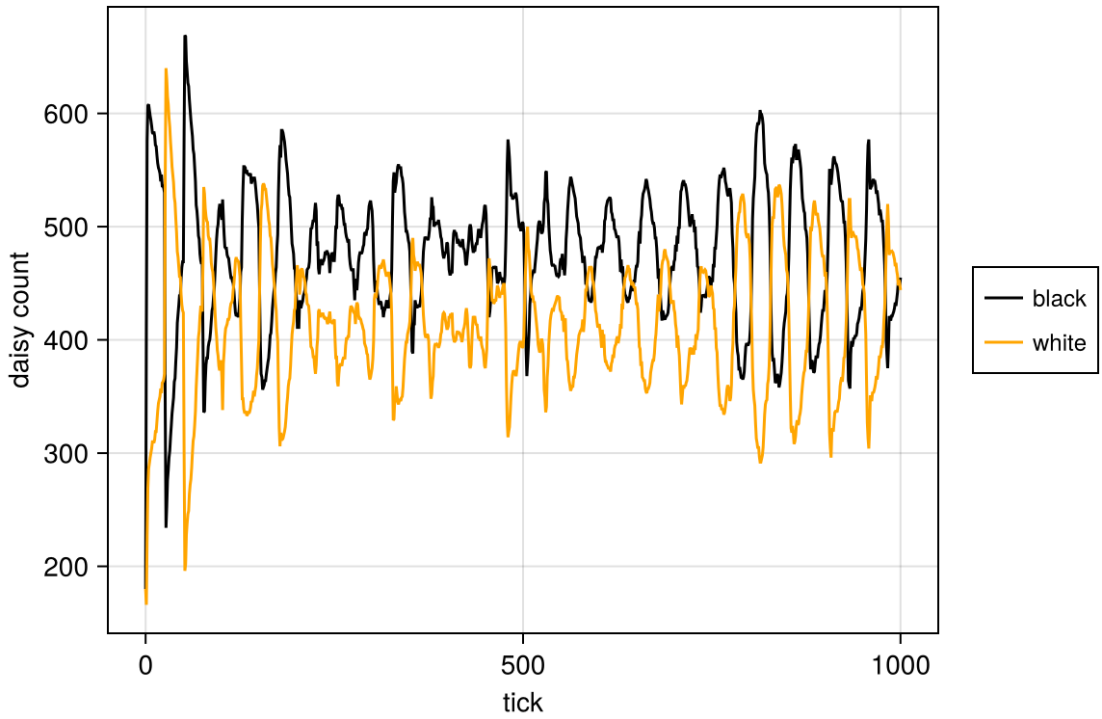
3.5 5. Динамика числа маргариток

Скрипт:

```
using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots



include(srcdir("daisyworld.jl"))

using CairoMakie

black(a) = a.breed == :black
white(a) = a.breed == :white
adata = [(black, count), (white, count)]

model = daisyworld(; solar_luminosity = 1.0)
```

```
agent_df, model_df = run!(model, 1000; adata)
figure = Figure(size = (600, 400));
ax = figure[1, 1] = Axis(figure, xlabel = "tick", ylabel = "daisy count")
blackl = lines!(ax, agent_df[!, :time], agent_df[!, :count_black], color = :black)
whitel = lines!(ax, agent_df[!, :time], agent_df[!, :count_white], color = :orange)
Legend(figure[1, 2], [blackl, whitel], ["black", "white"], labelsize = 12)
# figure

save(plotsdir("daisy_count.png"), figure)
```

Результат:



Анализ:

- сначала быстрый рост популяции
- затем колебания
- система выходит на устойчивый режим

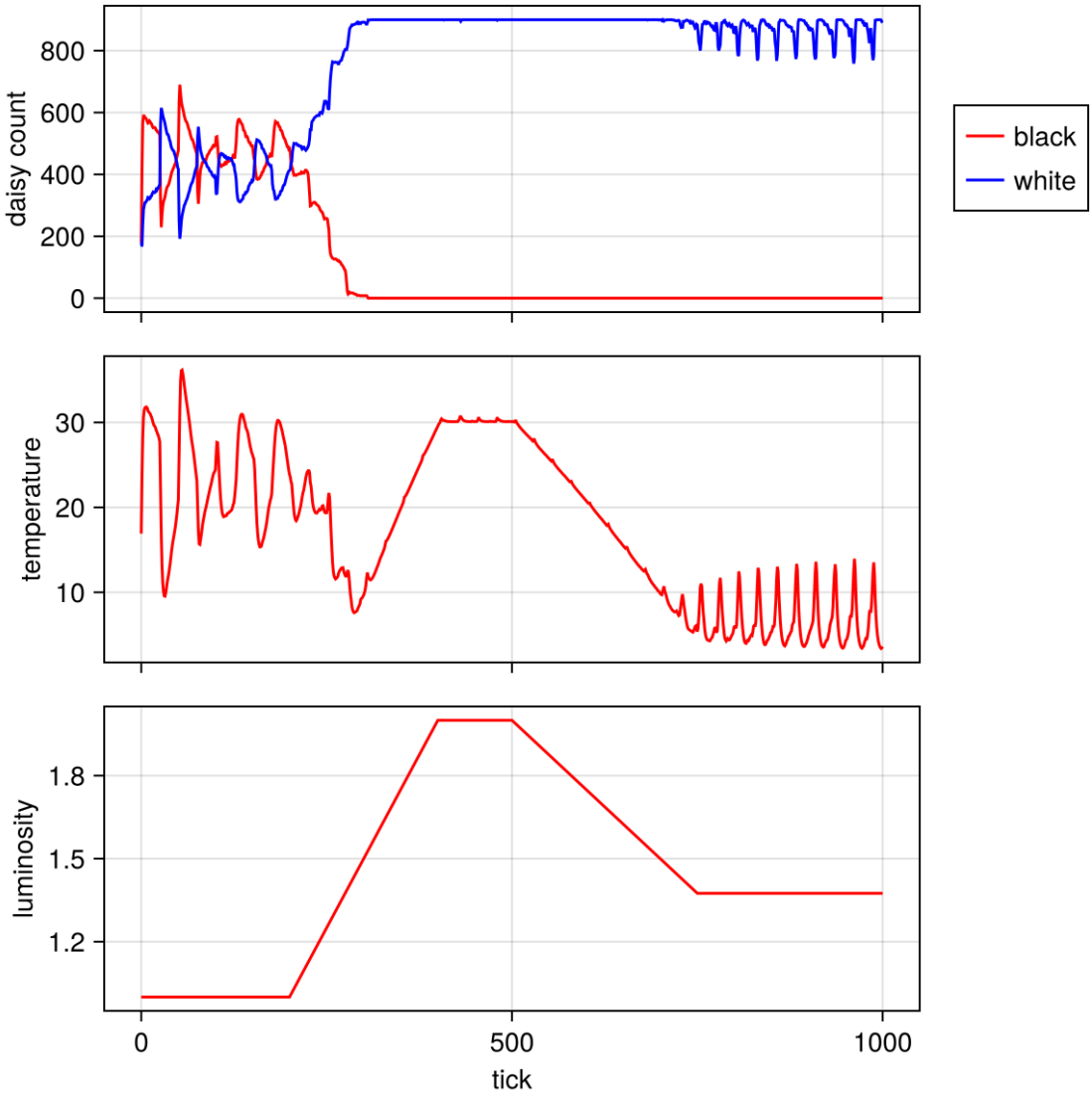
3.6 6. Динамика модели

Скрипт:

```
using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots



include(srcdir("daisyworld.jl"))

using CairoMakie

black(a) = a.breed == :black
white(a) = a.breed == :white
adata = [(black, count), (white, count)]

model = daisyworld(solar_luminosity = 1.0, scenario = :ramp)

temperature(model) = StatsBase.mean(model.temperature)
mdata = [temperature, :solar_luminosity]

agent_df, model_df = run!(model, 1000; adata = adata, mdata = mdata)

figure = CairoMakie.Figure(size = (600, 600));
ax1 = figure[1, 1] = Axis(figure, ylabel = "daisy count")
blackl = lines!(ax1, agent_df[!, :time], agent_df[!, :count_black], color = :red)
```

```
whitel = lines!(ax1, agent_df[!, :time], agent_df[!, :count_white], color = :blue)
figure[1, 2] = Legend(figure, [blackl, whitel], ["black", "white"])

ax2 = figure[2, 1] = Axis(figure, ylabel = "temperature")
ax3 = figure[3, 1] = Axis(figure, xlabel = "tick", ylabel = "luminosity")
lines!(ax2, model_df[!, :time], model_df[!, :temperature], color = :red)
lines!(ax3, model_df[!, :time], model_df[!, :solar_luminosity], color = :red)
for ax in (ax1, ax2); ax.xticklabelsvisible = false; end
figure


save(plotsdir("daisy_luminosity.png"), figure)
```

Результат:



Анализ:

- при изменении светимости система адаптируется
- белые маргаритки доминируют при высокой температуре
- происходит саморегуляция

3.7 7. Параметрические эксперименты

Скрипты:

```julia
using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots
using CairoMakie

include(srcdir("daisyworld.jl"))

## Параметры эксперимента
param_dict = Dict(
    :griddims => (30, 30),
    :max_age => [25, 40],
    :init_white => [0.2, 0.8],
    :init_black => 0.2,
    :albedo_white => 0.75,
    :albedo_black => 0.25,
    :surface_albedo => 0.4,
    :solar_change => 0.005,
    :solar_luminosity => 1.0,
    :scenario => :default,
    :seed => 165,
)

## Создаём список всех комбинаций
params_list = dict_list(param_dict)

for params in params_list
```

```
    model = daisyworld(;params...)

    daisycolor(a::Daisy) = a.breed

    plotkwargs = (
        agent_color=daisycolor,
        agent_size = 20,
        agent_marker = '□',
        heatarray = :temperature,
        heatkwargs = (colorrange = (-20, 60),),
    )
    plt1, _ = abmplot(model; plotkwargs...)

    step!(model, 5)
    plt2, _ = abmplot(model; heatarray = model.temperature, plotkwargs...)

    step!(model, 40)
    plt3, _ = abmplot(model; heatarray = model.temperature, plotkwargs...)

    plt1_name = savename("daisyworld",params) * "_step01" * ".png"
    plt2_name = savename("daisyworld",params) * "_step04" * ".png"
    plt3_name = savename("daisyworld",params) * "_step40" * ".png"

    save(plotsdir(plt1_name), plt1)
    save(plotsdir(plt2_name), plt2)
    save(plotsdir(plt3_name), plt3)

end
```

```julia
using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots
using CairoMakie

include(srcdir("daisyworld.jl"))


black(a) = a.breed == :black
white(a) = a.breed == :white
adata = [(black, count), (white, count)]

## Параметры эксперимента
param_dict = Dict(
    :griddims => (30, 30),
    :max_age => [25, 40],
    :init_white => [0.2, 0.8],
    :init_black => 0.2,
    :albedo_white => 0.75,
    :albedo_black => 0.25,
    :surface_albedo => 0.4,
    :solar_change => 0.005,
    :solar_luminosity => 1.0,
    :scenario => :default,
    :seed => 165,
```

```
)

## Создаём список всех комбинаций
params_list = dict_list(param_dict)

for params in params_list

    model = daisyworld(;params...)

    agent_df, model_df = run!(model, 1000; adata)
    figure = Figure(size = (600, 400));
    ax = figure[1, 1] = Axis(figure, xlabel = "tick", ylabel = "daisy count")
    blackl = lines!(ax, agent_df[!, :time], agent_df[!, :count_black], color = :black
    whitel = lines!(ax, agent_df[!, :time], agent_df[!, :count_white], color = :orang
    Legend(figure[1, 2], [blackl, whitel], ["black", "white"], labelsize = 12)

    plt_name = savename("daisy-count",params) * ".png"

    save(plotsdir(plt_name), figure)

end


using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots
```

```julia
using CairoMakie

include(srcdir("daisyworld.jl"))


black(a) = a.breed == :black
white(a) = a.breed == :white
adata = [(black, count), (white, count)]

## Параметры эксперимента
param_dict = Dict(
    :griddims => (30, 30),
    :max_age => [25, 40],
    :init_white => [0.2, 0.8],
    :init_black => 0.2,
    :albedo_white => 0.75,
    :albedo_black => 0.25,
    :surface_albedo => 0.4,
    :solar_change => 0.005,
    :solar_luminosity => 1.0,
    :scenario => :ramp,
    :seed => 165,
)

## Создаём список всех комбинаций
params_list = dict_list(param_dict)

for params in params_list
```

```
    model = daisyworld(;params...)

    temperature(model) = StatsBase.mean(model.temperature)
    mdata = [temperature, :solar_luminosity]

    agent_df, model_df = run!(model, 1000; adata = adata, mdata = mdata)

    figure = CairoMakie.Figure(size = (600, 600));
    ax1 = figure[1, 1] = Axis(figure, ylabel = "daisy count")
    blackl = lines!(ax1, agent_df[!, :time], agent_df[!, :count_black], color = :red)
    whitel = lines!(ax1, agent_df[!, :time], agent_df[!, :count_white], color = :blue
    figure[1, 2] = Legend(figure, [blackl, whitel], ["black", "white"])

    ax2 = figure[2, 1] = Axis(figure, ylabel = "temperature")
    ax3 = figure[3, 1] = Axis(figure, xlabel = "tick", ylabel = "luminosity")
    lines!(ax2, model_df[!, :time], model_df[!, :temperature], color = :red)
    lines!(ax3, model_df[!, :time], model_df[!, :solar_luminosity], color = :red)
    for ax in (ax1, ax2); ax.xticklabelsvisible = false; end


    plt_name = savename("daisy-luminosity",params) * ".png"
    save(plotsdir(plt_name), figure)

end
```

Примеры результатов:

```
readdir(plotsdir())
```

Анализ:

- увеличение `init_white` → охлаждение системы
- увеличение `max_age` → стабилизация
- параметры существенно влияют на динамику

4 Общий анализ

Модель демонстрирует:

- устойчивость системы
- адаптацию к изменениям среды
- баланс между агентами

Это подтверждает гипотезу о саморегуляции системы.

5 Подтверждение выполнения (скриншоты)

В данном разделе приведены скриншоты выполнения лабораторной работы.

5.1 Установка зависимостей

```
julia> using Pkg

julia> Pkg.add(["Agents", "CairoMakie", "DataFrames", "StatsBase", "Plots
atson"])
    Updating registry at `~/.julia/registries/General.toml`
   Resolving package versions...
   Installed ImageIO ──────────────────── v0.6.9
   Installed AdaptivePredicates ─────────── v1.2.0
   Installed OffsetArrays ───────────────── v1.17.0
   Installed JpegTurbo ──────────────────── v0.1.6
   Installed Unitful ────────────────────── v1.28.0
   Installed PNGFiles ───────────────────── v0.4.4
   Installed AxisArrays ─────────────────── v0.4.8
```

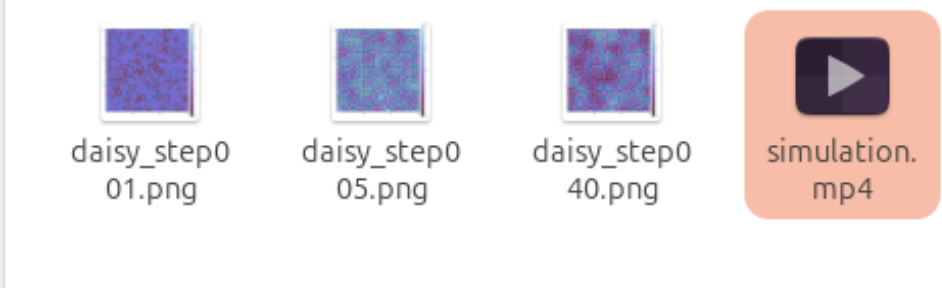
5.2 Запуск модели

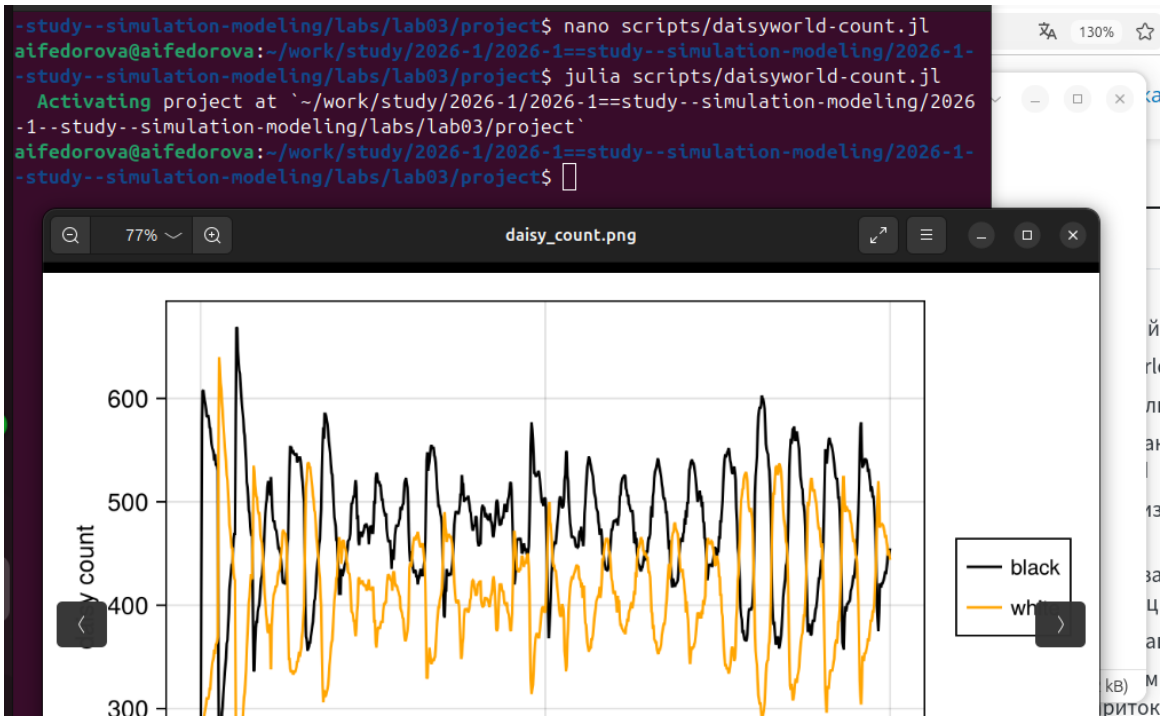
```
aifedorova@aifedorova:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1
-study--simulation-modeling/labs/lab03/project$ nano src/daisyworld.jl
aifedorova@aifedorova:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1
-study--simulation-modeling/labs/lab03/project$ nano scripts/daisyworld.jl
aifedorova@aifedorova:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1
-study--simulation-modeling/labs/lab03/project$ julia scripts/daisyworld.jl
  Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/202
-1--study--simulation-modeling/labs/lab03/project`
```

5.3 Генерация анимации

```
-study--simulation-modeling/labs/lab03/project$ julia scripts/daisyworld-animate.jl
  Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026
-1--study--simulation-modeling/labs/lab03/project`
aifedorova@aifedorova:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1-
-study--simulation-modeling/labs/lab03/project$
```

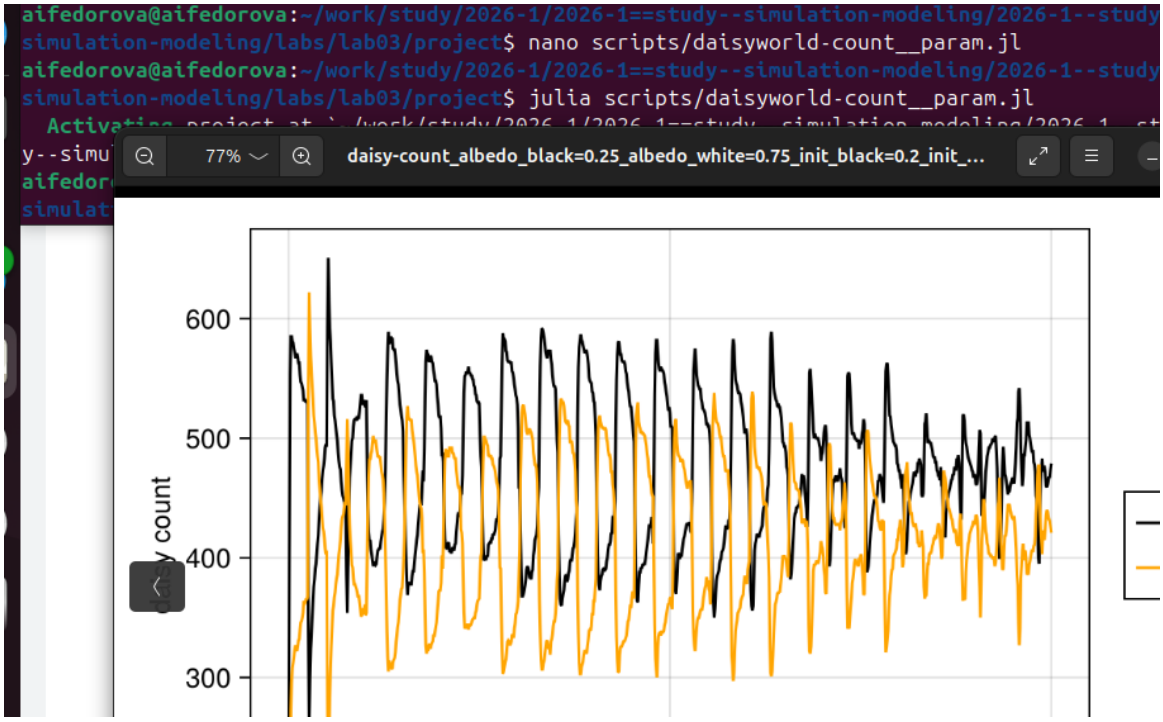
5.4 Построение графика численности





5.5 Построение комплексного графика

5.6 Параметрические эксперименты



aifedorova@aifedorova:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab03/project$ nano scripts/daisyworld-luminosity__param.jl
aifedorova@aifedorova:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab03/project$ julia scripts/daisyworld-luminosity__param.jl
Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab03/project`
aifedorova@aifedorova:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--
27%
daisy-luminosity_albedo_black=0.25_albedo_white=0.75_init_black=0.2_init_white=0.8_max...
daisy count
black
white
temperature
luminosity
tick

6 Выводы

В ходе лабораторной работы:

- реализована агентная модель Daisyworld
- выполнена визуализация и анализ
- исследовано влияние параметров
- подтверждён эффект саморегуляции

7 Список литературы

1. Watson A.J., Lovelock J.E.
2. Datseris G. Agents.jl
3. Wood A.J. Daisyworld review